1. **The post order traversal of a binary tree is 8,9,6,7,4,5,2,3,1. The inorder traversal of the same tree is 8, 6,9, 4,7,2,5, 1, 3. The height of a tree is the length of the longest path from the root to any leaf. The height of the binary tree above is**

The height of the binary tree is **4**.

In this context, height is defined as the length of the longest path from the root to any leaf node, measured by the number of edges.

1. Identify the Root

In a postorder traversal (Left-Right-Root), the last element is always the root of the tree.

- **Postorder:** $8, 9, 6, 7, 4, 5, 2, 3, \mathbf{1}$
- **Root node:** 1

2. Determine Subtrees

Using the inorder traversal (Left-Root-Right), locate the root to split the tree into left and right subtrees.

- **Inorder:** $(8, 6, 9, 4, 7, 2, 5), \mathbf{1}, (3)$
- **Left Subtree nodes:** $\{8, 6, 9, 4, 7, 2, 5\}$
- **Right Subtree nodes:** $\{3\}$

3. Recursive Construction

Repeat this process for the left subtree to determine its structure:

- **New Root (Left Subtree):** The last element in the corresponding postorder segment $\{8, 9, 6, 7, 4, 5, 2\}$ is **2**.
- **Split at 2:** Inorder $\{8, 6, 9, 4, 7\}, \mathbf{2}, \{5\}$.
- **New Root (Sub-left):** Last of $\{8, 9, 6, 7, 4\}$ is **4**.
- **Split at 4:** Inorder $\{8, 6, 9\}, \mathbf{4}, \{7\}$.
- **New Root (Sub-sub-left):** Last of $\{8, 9, 6\}$ is **6**.
- **Split at 6:** Inorder $\{8\}, \mathbf{6}, \{9\}$.

4. Trace the Longest Path

Following the construction above, the longest path extends from the root (1) through the left children:

- **Path:** $1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 8$ (or $6 \rightarrow 9$)
- **Edges:** 1 (to 2) + 1 (to 4) + 1 (to 6) + 1 (to 8) = **4 edges**.

Result

The height of the binary tree is **4**.

2. **https://www.wscubetech.com/resources/dsa**

3. **What can be said about the array representation of a circular queue when it contains only one element?**

In the array representation of a **circular queue**, when the queue contains **only one element**:

$$\boxed{\text{FRONT} = \text{REAR}}$$

Explanation

- `FRONT` points to the first element.
- `REAR` points to the last element.
- If there is only one element, that element is both the first and the last.

Therefore, both indices become equal.

This condition is commonly used to identify a single-element circular queue.

4. **+ A* - BCD is a prefix expression. If A, B, C, D have value 5,4,2,3 respectively the expression evaluates to**

Given:

- $A = 5$
- $B = 4$
- $C = 2$
- $D = 3$

Step 1: Evaluate $- \ B \ C$

$$4 - 2 = 2$$

Step 2: Evaluate $* \ 2 \ D$

$$2 \times 3 = 6$$

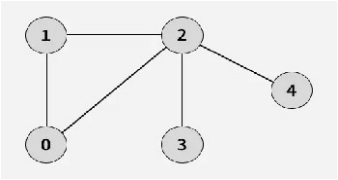
Step 3: Evaluate $+ \ A \ 6$

$$5 + 6 = 11$$

Final Answer

$$\boxed{11}$$

5. **Breadth First Search**



Output: *[0, 1, 2, 3, 4]*

Explanation: *Starting from 0, the BFS traversal proceeds as follows:*

Visit 0 - Print: 0

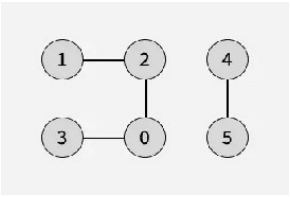
Visit 1 (neighbor of 0) - Print: 1

Visit 2 (next neighbor of 0) - Print: 2

Visit 3 (first neighbor of 2 that hasn't been visited yet) - Print: 3

Visit 4 (next neighbor of 2) - Print: 4

6.



Output: *[0, 2, 3, 1, 4, 5]*

Explanation: *Starting from 0, the BFS traversal proceeds as follows:*

Visit 0 - Print: 0

Visit 2 (first neighbor of 0) - Print: 2

Visit 3 (next neighbor of 0) - Print: 3

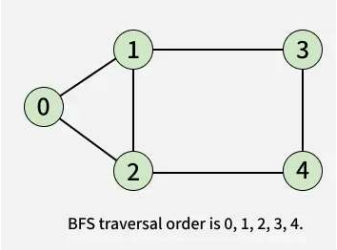
Visit 1 (neighbor of 2 that hasn't been visited yet) - Print: 1

Start another BFS traversal with source as 4:

Visit 4 - Print: 4

Visit 5 (neighbor of 4) - Print: 5

7.



8. **Which of the following can be the sequence of nodes examined in binary search tree while searching for key 88?**

90, 40, 65, 50, 88

90, 110, 80, 85, 88

190, 60, 90, 85, 88

65, 140, 80, 70, 88

The sequence that can be the nodes examined in a **Binary Search Tree (BST)** while searching for the key **88** is:

90, 40, 65, 50, 88 ✗ (Incorrect)
90, 110, 80, 85, 88 ✗ (Incorrect)
190, 60, 90, 85, 88 ✓ (Correct)
65, 140, 80, 70, 88 ✗ (Incorrect)

Explanation of the Correct Option

In a **Binary Search Tree**, for any node u, all keys in the left subtree must be smaller than u, and all keys in the right subtree must be larger than u. The sequence **190, 60, 90, 85, 88** follows these rules:

1. **190**: Start at root. Since $88 < 190$, go left. All subsequent nodes must be < 190.
2. **60**: Since $88 > 60$, go right. All subsequent nodes must be > 60. (Range: 61 to 189)
3. **90**: Since $88 < 90$, go left. All subsequent nodes must be < 90. (Range: 61 to 89)
4. **85**: Since $88 > 85$, go right. All subsequent nodes must be > 85. (Range: 86 to 89)
5. **88**: Key found.

Why the Other Options are Incorrect

- **90, 40, 65, 50, 88**: After visiting **65**, we look for **88**. Since $88 > 65$, we go right. Every node in that subtree must be > 65. However, the next node is **50**, which violates the BST property.
- **90, 110, 80, 85, 88**: After visiting **110**, we look for **88**. Since $88 < 110$, we go left. Every node in that subtree must be < 110. But we already know $88 < 90$ from the first step; however, once we move to **110**, we are in the right subtree of **90**, meaning all nodes must be > 90. **80** is less than **90**, which is a violation.
- **65, 140, 80, 70, 88**: After visiting **140**, we go left to find **88**. Every node must be < 140. After visiting **80**, we go right to find **88** (so nodes must be > 80). The next node **70** is smaller than **80**, violating the BST property. solutions adda +4

9. **If we want to find the last node of a linked list, then the correct coding is**

```
while (temp->next != NULL) { temp = temp->next; }
```

10. **To check whether the linked list is empty**

```
if (head == nullptr) { // List is empty }
else { // Access head }
```

11. **// Count nodes in linked list**

```
while (head) {
  length++;
  head = head->next;
}
```

Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	Ω(n log(n))	θ(n log(n))	O(n^2)	O(log(n))
Mergesort	Ω(n log(n))	θ(n log(n))	O(n log(n))	O(n)
Timsort	Ω(n)	θ(n log(n))	O(n log(n))	O(n)
Heapsort	Ω(n log(n))	θ(n log(n))	O(n log(n))	O(1)
Bubble Sort	Ω(n)	θ(n^2)	O(n^2)	O(1)
Insertion Sort	Ω(n)	θ(n^2)	O(n^2)	O(1)
Selection Sort	Ω(n^2)	θ(n^2)	O(n^2)	O(1)
Tree Sort	Ω(n log(n))	θ(n log(n))	O(n^2)	O(n)
Shell Sort	Ω(n log(n))	θ(n(log(n))^2)	O(n(log(n))^2)	O(1)
Bucket Sort	Ω(n+k)	θ(n+k)	O(n^2)	O(n)
Radix Sort	Ω(nk)	θ(nk)	O(nk)	O(n+k)
Counting Sort	Ω(n+k)	θ(n+k)	O(n+k)	O(k)
Cubesort	Ω(n)	θ(n log(n))	O(n log(n))	O(n)

12.

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	θ(1)	θ(n)	θ(n)	θ(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Stack	θ(n)	θ(n)	θ(1)	θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	θ(n)	θ(n)	θ(1)	θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Singly-Linked List	θ(n)	θ(n)	θ(1)	θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly-Linked List	θ(n)	θ(n)	θ(1)	θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Skip List	θ(log(n))	θ(log(n))	θ(log(n))	θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
Hash Table	N/A	θ(1)	θ(1)	θ(1)	N/A	O(n)	O(n)	O(n)	O(n)
Binary Search Tree	θ(log(n))	θ(log(n))	θ(log(n))	θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	N/A	θ(log(n))	θ(log(n))	θ(log(n))	N/A	O(n)	O(n)	O(n)	O(n)
B-Tree	θ(log(n))	θ(log(n))	θ(log(n))	θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Red-Black Tree	θ(log(n))	θ(log(n))	θ(log(n))	θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Splay Tree	N/A	θ(log(n))	θ(log(n))	θ(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))	O(n)
AVL Tree	θ(log(n))	θ(log(n))	θ(log(n))	θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
KD Tree	θ(log(n))	θ(log(n))	θ(log(n))	θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)

13. For merging two sorted lists of size m and n into a sorted list of size m+n, we required comparisons of

Merging two sorted lists of size m and n into a single sorted list of size $m + n$ requires a maximum of $\boldsymbol{m + n - 1}$ comparisons in the worst case. This indicates that the time complexity is $\boldsymbol{O(m + n)}$.

- **Best Case:** $\min(m, n)$ comparisons.
- **Worst Case:** $m + n - 1$ comparisons.
- **Average Case:** $O(m + n)$.

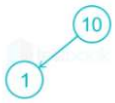
The algorithm functions by comparing the smallest remaining elements of each list and adding the smaller one to the final list, continuing until one list is empty, then appending the remainder.

14. The following numbers are inserted into an empty binary search tree in the given order: 10,1, 3, 5,15,12,16. What is the height of the binary search tree (the height is the maximum distance of a leaf node from the root)?

Explanation:

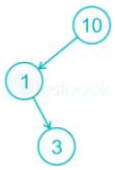
Step 1: First 10 comes and now that is the **Root** node.

Step 2: Now 1 came and 1 < 10 then insert Node 1 to the **Left** of Node 10.



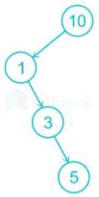
Step 3: Now 3 came and 3 < 10 go to the **Left** of

Node 10 and check 3 > 1 then insert Node 3 to the **Right** of Node 1.

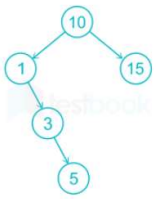


Step 4: Now 5 came and 5 < 10 go to the **Left** of

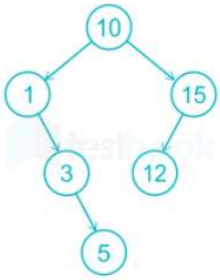
Node 10 and check 5 > 1 go to the **Right** of Node 1 then check 5 > 3 then insert Node 5 to the **Right** of Node 3.



Step 5: Now 15 came and 15 > 10 then insert Node 15 to the **Right** of Node 10.

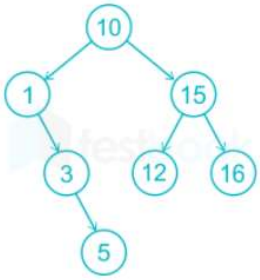


Step 6: Now 12 came and 12 > 10 go to the **Right** of Node 10 and check 15 > 12 then insert Node 12 to the **Left** of Node 15.



Step 7: Now 16 came and 16 > 10 go to the **Right** of

10 and check 16 > 15 then insert 16 to the **Right** of Node 15.



After step 7, we can count the height of the tree as 3.

15. The maximum number of binary trees that can be formed with three unlabeled nodes is

2* nCn/(n+1) = 2* 3C3/(3+1) = 5